

CHAPTER 1

Introduction

1.1 Overview

With the rise of digital payments and the rapid growth of online shopping, credit card fraud has become a significant concern for both financial institutions and consumers. One of the most notable trends in recent years has been the surge in card-not-present fraud, which has been accelerated by the global pandemic and the corresponding increase in e-commerce. As consumers moved to online shopping during the COVID-19 pandemic, fraudsters took advantage of the situation, resulting in a 35% increase in online fraud in 2020 alone. This fraud occurs when stolen credit card details are used for unauthorized transactions, typically in online settings.

Another troubling trend is the increasing prevalence of synthetic identity fraud, where fraudsters create fake identities using a mix of real and fabricated personal information. This type of fraud is difficult to detect by traditional methods, and it is projected to cause over \$6 billion in losses globally. Financial institutions are facing growing challenges in combating this, as synthetic identities can bypass conventional fraud detection systems.

Alongside these types of fraud, mobile payment fraud has also risen sharply. As mobile wallets such as Apple Pay, Google Pay, and Samsung Pay become more popular, fraudsters are increasingly targeting these platforms. Mobile payment fraud is expected to account for over \$100 billion in losses globally by 2023, making it a major concern for the financial industry.

Data breaches remain a significant source of fraud, with hackers stealing millions of credit card details and using them for account takeover fraud. As data breaches continue to affect large corporations and e-commerce platforms, fraudsters exploit the stolen data to gain access to users' accounts and make unauthorized purchases. In 2021, breaches affected millions of people, increasing the likelihood of identity theft and fraud.

In response to these rising threats, machine learning (ML) and artificial intelligence (AI) have become essential tools for detecting and preventing fraud. Traditional fraud detection methods, which are often based on rule-based systems, are no longer sufficient to handle the complexity

of modern fraud techniques. ML and AI can analyse vast amounts of transaction data in real time, identify suspicious patterns, and adapt to new fraud tactics. This advanced technology has significantly improved the accuracy and speed of fraud detection, enabling financial institutions to respond more effectively to fraudulent activities.

Additionally, there has been a growing adoption of multi-factor authentication (MFA) across financial platforms to enhance security. MFA combines multiple verification methods, such as passwords, biometric data, and one-time codes, to ensure that only legitimate users can complete transactions. This approach is helping reduce the risk of fraud and adding an additional layer of protection for users.

Finally, real-time fraud detection is gaining traction as financial institutions move away from post-transaction analysis. By leveraging AI and ML algorithms, these systems can now detect fraud in real time, preventing fraudulent transactions before they are completed. This shift toward real-time monitoring represents a significant advancement in fraud prevention, allowing financial institutions to protect customers and minimize financial losses more effectively.

In conclusion, the recent trends in credit card fraud highlight the evolving tactics used by criminals and the growing need for sophisticated fraud detection systems. As fraud continues to rise, particularly in online and mobile payment spaces, financial institutions must adopt more advanced technologies, such as AI and ML, to stay ahead of emerging threats and protect both consumers and businesses from the financial impact of fraud.

1.2 Objective

The primary objective of deploying credit card fraud detection systems is to safeguard financial transactions by identifying and preventing fraudulent activities in real time. By leveraging machine learning algorithms, these systems aim to minimize financial losses, protect customer trust, and ensure operational efficiency in handling transactions.

1. Enhance Accuracy: Utilize advanced machine learning models to detect fraudulent transactions with high precision, reducing false positives and negatives that could disrupt legitimate user experiences.

2. Real-Time Detection: Enable instant identification and prevention of fraudulent transactions to mitigate financial losses and maintain customer trust in financial systems.

3. Handle Imbalanced Data: Address the challenge of fraud being a rare event in large datasets by employing techniques like resampling or synthetic data generation to improve detection performance.

4. Ensure Scalability: Develop models capable of handling high volumes of transactional data in real-time environments, such as those seen in e-commerce platforms and banking institutions.

6. Minimize Fraudulent Losses: Reduce financial damages by promptly identifying fraudulent activities and stopping them before they can impact users or organizations.

7. Improve Operational Efficiency: Streamline fraud monitoring and investigation processes by automating the detection of suspicious patterns, reducing the need for extensive manual reviews.

8. Support Compliance and Security Standards: Provide accurate, reliable fraud detection that aligns with industry regulations and enhances the security framework of financial systems.

9. Adapt to Evolving Threats: Continuously improve fraud detection capabilities by incorporating adaptive machine learning models that can learn and respond to new fraud patterns as they emerge.

CHAPTER 2

Literature Review

1.1 Introduction

Credit card fraud detection is a critical task for financial institutions to protect customers from unauthorized transactions. With the increasing volume of credit card usage globally, traditional methods are becoming ineffective, and machine learning (ML) techniques have become integral for real-time fraud detection. By utilizing models such as Logistic Regression, Decision Trees, Random Forests and Gradient Boosting, financial institutions can enhance their fraud detection capabilities, identify fraudulent transactions quickly, and improve overall security.

1.2 Literature Review

Title	Authors	Year	Objective	Methodology	Limitations
Credit Card Fraud Detection Using Machine Learning	Adepoju, O., Wosowei, J., Lawte, S., & Jaiman, H.	2019	To compare the effectiveness of KNN, SVM, and Logistic Regression in detecting fraudulent transactions.	Applied K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Logistic Regression on a credit card transaction dataset to classify transactions as fraudulent or non-fraudulent.	The study was conducted on a specific dataset, which may limit the generalizability of the results to other datasets or real-world scenarios.
A Machine Learning Based Credit	Ryman-Tubb, N. F.,	2022	To develop a credit card fraud	Utilized a genetic algorithm for feature selection	The approach may be computationally

Card Fraud Detection Using the GA	Krause, P., & Garn, W.		detection model using machine learning with genetic algorithm-based feature selection.	combined with machine learning classifiers to improve fraud detection accuracy.	intensive due to the genetic algorithm, and the performance is highly dependent on the quality of feature selection.
Securing Transactions: A Hybrid Dependable Ensemble Machine Learning Model using IHT-LR and Grid Search	Talukder, M. A., Hossen, R., Uddin, M. A., Uddin, M. N., & Acharjee, U.	2024	To enhance fraud detection by combining multiple algorithms with weighted optimization.	Proposed a hybrid ensemble model integrating Decision Tree, Random Forest, K-Nearest Neighbor, and Multilayer Perceptron, optimized using Grid Search and Instant Hardness Threshold with Logistic Regression.	The complexity of the ensemble model may lead to overfitting, and the approach requires extensive computational resources for training and optimization.
Ensemble and Mixed Learning Techniques for Credit Card Fraud Detection	de Souza, D. H. M., & Bordin Jr, C. J.	2021	To improve fraud detection performance using ensemble and mixed	Implemented a mixed learning technique with K-means preprocessing followed by classification, and an adapted	The methods may reduce computational cost but could be less effective in detecting novel fraud patterns not

			learning techniques.	detector ensemble technique using OR-logic algorithm aggregation.	present in the training data.
Credit Card Fraud Detection with Subspace Learning-based One-Class Classification	Zaffar, Z., Sohrab, F., Kannaiainen, J., & Gabbouj, M.	2023	To address high-dimensional and imbalanced data challenges in fraud detection.	Investigated subspace learning-based One-Class Classification algorithms to handle imbalanced data distributions and anticipate unknown fraud techniques.	The approach may not effectively detect fraud types that do not conform to learned patterns, and the subspace learning process can be complex.

CHAPTER 3

Project Description

3.1 Introduction

To achieve the goal of detecting credit card fraud effectively and identifying the best-performing model, several systematic steps are undertaken. The process begins with selecting an appropriate dataset and applying preprocessing techniques to prepare it for modelling. Following this, machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting are implemented. These models are trained and evaluated using relevant performance metrics. The ultimate aim is to compare the models and select the one that provides the highest accuracy and reliability for detecting fraudulent transactions.

3.2 Data Source

The dataset utilized in this project was sourced from an open-access repository, Kaggle.com. It contains anonymized credit card transaction data collected over two days in September 2013, representing transactions made by European cardholders. The dataset consists of 284,807 rows and 31 attributes. Of these attributes, 28 are numeric variables transformed using Principal Component Analysis (PCA) to maintain customer confidentiality. The remaining three attributes are:

- **Time:** Represents the elapsed time (in seconds) between the first transaction and each subsequent transaction.
- **Amount:** Indicates the transaction amount, useful for feature analysis.
- **Class:** A binary variable where 1 indicates a fraudulent transaction and 0 indicates a legitimate transaction.

The dataset is highly imbalanced, with the fraudulent cases accounting for only a small fraction of the total transactions, which makes it an ideal choice for evaluating fraud detection methods.

Dataset Link: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

CHAPTER 4

Data Analysis and Preprocessing

4.1 Data Analysis

The first figure below shows the structure of the dataset where all attributes are shown, with their type, in addition to a glimpse of the variables within each Attribute; as shown at the end of the figure, the Class type is integer, which I needed to change to factor and identify the 0 as Not Fraud and the one as Fraud to ease the process of creating the model and obtain visualizations. The second figure shows the class distribution; the red bar, which contains 284,315 variables, represents the non-fraudulent transactions, and the blue bar, with 492 variables, represents the fraudulent transactions.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Time        284807 non-null  float64
1    V1          284807 non-null  float64
2    V2          284807 non-null  float64
3    V3          284807 non-null  float64
4    V4          284807 non-null  float64
5    V5          284807 non-null  float64
6    V6          284807 non-null  float64
7    V7          284807 non-null  float64
8    V8          284807 non-null  float64
9    V9          284807 non-null  float64
10   V10         284807 non-null  float64
11   V11         284807 non-null  float64
12   V12         284807 non-null  float64
13   V13         284807 non-null  float64
14   V14         284807 non-null  float64
15   V15         284807 non-null  float64
16   V16         284807 non-null  float64
17   V17         284807 non-null  float64
18   V18         284807 non-null  float64
19   V19         284807 non-null  float64
20   V20         284807 non-null  float64
21   V21         284807 non-null  float64
22   V22         284807 non-null  float64
23   V23         284807 non-null  float64
24   V24         284807 non-null  float64
25   V25         284807 non-null  float64
26   V26         284807 non-null  float64
27   V27         284807 non-null  float64
28   V28         284807 non-null  float64
29   Amount      284807 non-null  float64
30   Class       284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
None
```

Fig:4.1.1 Data Structure

The feature set included 31 attributes: 28 anonymized features transformed using Principal Component Analysis (PCA) for privacy reasons, and three original features—Time, Amount, and Class.

- **Time Feature:** This attribute represented the seconds elapsed since the first transaction and was used to identify temporal patterns. It was found to have no direct correlation with fraud and was retained for potential contextual relevance.
- **Amount Feature:** The transaction amount varied widely and was highly skewed, necessitating scaling. Analysis revealed no strong association between transaction size and fraud, though certain outliers warranted attention.

```
print(data.Amount.describe())
```

count	284807.000000
mean	88.349619
std	250.120109
min	0.000000
25%	5.600000
50%	22.000000
75%	77.165000
max	25691.160000
Name: Amount, dtype: float64	

Fig: 4.1.2 Amount Description

- **Class Distribution:** The target variable Class was significantly imbalanced, with 99.83% of the transactions labelled as legitimate (0) and only 0.17% labelled as fraudulent (1). This imbalance required addressing through sampling techniques to avoid biased model performance.

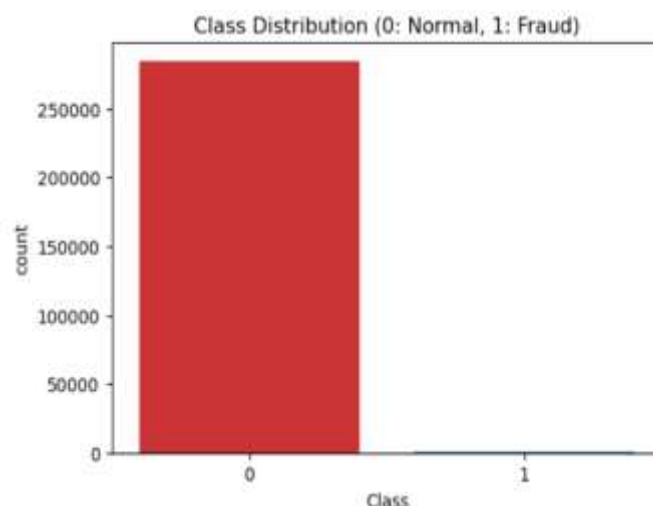


Fig:4.1.2 Class Distribution

Visualizations such as class distribution histograms illustrated the stark imbalance, emphasizing the need for careful handling during training. Correlation heatmaps showed

minimal direct relationships between features, likely due to the PCA transformation. Exploratory analysis of individual features revealed clusters and potential anomalies that could indicate fraudulent behaviour.

```
print(data.describe())
```

	Time	V1	V2	V3	V4
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	3.918649e-15	5.682686e-16	-8.761736e-15	2.811118e-15
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01

Fig:4.1.3 Data Description

Through this analysis, we identified key characteristics of the dataset, guided the preprocessing steps, and highlighted the challenges posed by the imbalance. This groundwork ensured that our models were tailored to the nuances of the dataset, maximizing their potential to detect fraudulent transactions.

4.2 Data Preprocessing

Preprocessing the dataset is a critical step in preparing for accurate credit card fraud detection. The dataset used in this project is highly imbalanced, with only 0.17% fraudulent transactions compared to 99.83% legitimate transactions. This imbalance necessitated specific techniques to ensure fair training and evaluation of the models.

First, the features Time and Amount were scaled using StandardScaler to align their scale with the PCA-transformed features, which were already normalized. No further adjustments were needed for the 28 PCA-transformed features. The target variable Class, which indicates fraud (1) or legitimate transactions (0), was retained as the primary classification target.

```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(data.drop(['Class'], axis=1))

# Create a new DataFrame with scaled features
scaled_data = pd.DataFrame(scaled_features, columns=data.columns[:-1])
scaled_data['Class'] = data['Class'] # Add the target variable back
```

Fig:4.2.1 Feature Scaling

The dataset was split into training and testing sets using an 80-20 split, maintaining the original class distribution. This ensured the models were evaluated under realistic conditions. For algorithms like Logistic Regression and Decision Tree, undersampling was applied to the majority class in the training set. By reducing the number of legitimate transactions, we created a balanced dataset that allowed these models to effectively learn from the minority class without being overwhelmed by the majority class.

```
: normal = data[data.Class==0]
  fraud = data[data.Class==1]

normal_sample = normal.sample(n=492)
new_dataset = pd.concat([normal_sample, fraud], axis=0)

# Splitting the dataset
X = new_dataset.drop(columns='Class', axis=1)
Y = new_dataset['Class']
```

Fig:4.2.2 Under sampling

In contrast, Random Forest and Gradient Boosting leveraged their inherent capability to handle class imbalance using weighted mechanisms, eliminating the need for explicit balancing. This versatility allowed them to operate on the imbalanced dataset while still achieving strong performance.

```
# Define features and target variable
X = scaled_data.drop(['Class'], axis=1)
y = scaled_data['Class']

# Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2, stratify=y)
```

Fig:4.2.3 Dataset Splitting

By carefully scaling, splitting, and balancing the data as appropriate, we ensured a robust preprocessing pipeline that prepared the dataset for effective training and testing across all algorithms. These steps were essential in mitigating the challenges posed by the imbalance and enabling reliable fraud detection.

CHAPTER 5

Algorithm And Performance Analysis

5.1 Logistic Regression

Logistic regression is used for binary classification where we use sigmoid function, that takes input as independent variables and produces a probability value between 0 and 1.

For example, we have two classes Class 0 and Class 1 if the value of the logistic function for an input is greater than 0.5 (threshold value) then it belongs to Class 1 otherwise it belongs to Class 0. It's referred to as regression because it is the extension of linear regression but is mainly used for classification problems.

Logistic regression predicts the output of a categorical dependent variable. Therefore, the outcome must be a categorical or discrete value.

It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).

5.1.1 Logistic Function – Sigmoid Function

The sigmoid function is a mathematical function used to map the predicted values to probabilities. It maps any real value into another value within a range of 0 and 1. The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function. In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

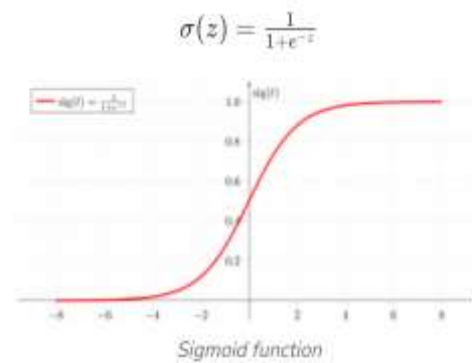


Fig:5.1.1 Sigmoid Function

5.1.2 Logistic Regression Equation

The odd is the ratio of something occurring to something not occurring. it is different from probability as the probability is the ratio of something occurring to everything that could possibly occur. so odd will be:

$$\frac{p(x)}{1-p(x)} = e^z$$

Applying natural log on odd. then log odd will be:

$$\begin{aligned} \log \left[\frac{p(x)}{1-p(x)} \right] &= z \\ \log \left[\frac{p(x)}{1-p(x)} \right] &= w \cdot X + b \\ \frac{p(x)}{1-p(x)} &= e^{w \cdot X + b} \quad \dots \text{Exponentiate both sides} \\ p(x) &= e^{w \cdot X + b} \cdot (1-p(x)) \\ p(x) &= e^{w \cdot X + b} - e^{w \cdot X + b} \cdot p(x) \\ p(x) + e^{w \cdot X + b} \cdot p(x) &= e^{w \cdot X + b} \\ p(x)(1 + e^{w \cdot X + b}) &= e^{w \cdot X + b} \\ p(x) &= \frac{e^{w \cdot X + b}}{1 + e^{w \cdot X + b}} \end{aligned}$$

then the final logistic regression equation will be:

$$p(X; b, w) = \frac{e^{w \cdot X + b}}{1 + e^{w \cdot X + b}} = \frac{1}{1 + e^{-w \cdot X + b}}$$

5.1.3 Performance Analysis

Logistic Regression is a linear model widely used for binary classification. In our study, under sampling was applied to balance the dataset for this algorithm. LR achieved an accuracy of 93.91% on the test set, with an F1-score of 93.62%, indicating a strong balance between precision and recall. While effective, it demonstrated limitations in handling complex relationships in the data. (The Confusion Matrix and ROC curve will be discussed on section 5.5)

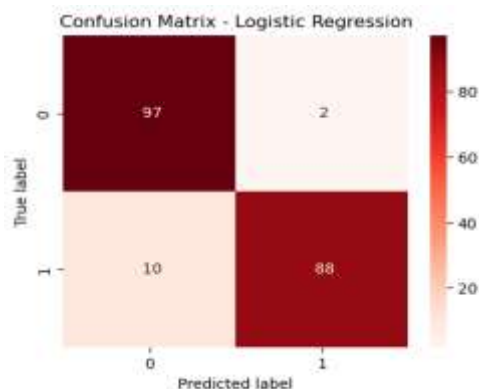


Fig:5.1.2 Confusion Matrix

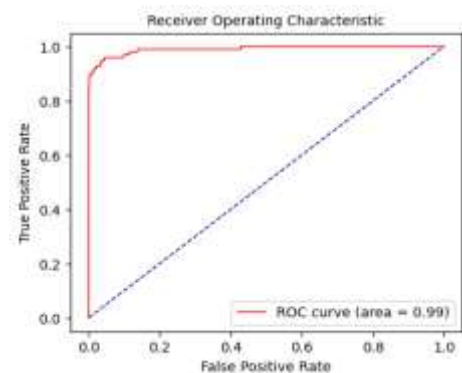


Fig:5.1.3 ROC Curve

5.2 Decision Tree

A decision tree is a flowchart-like structure used to make decisions or predictions. It consists of nodes representing decisions or tests on attributes, branches representing the outcome of these decisions, and leaf nodes representing final outcomes or predictions. Each internal node corresponds to a test on an attribute, each branch corresponds to the result of the test, and each leaf node corresponds to a class label or a continuous value.

5.2.1 Structure of Decision Tree

1. **Root Node:** Represents the entire dataset and the initial decision to be made.
2. **Internal Nodes:** Represent decisions or tests on attributes. Each internal node has one or more branches.
3. **Branches:** Represent the outcome of a decision or test, leading to another node.
4. **Leaf Nodes:** Represent the final decision or prediction. No further splits occur at these nodes.

5.2.2 How Decision Trees Work?

The process of creating a decision tree involves:

1. **Selecting the Best Attribute:** Using a metric like Gini impurity, entropy, or information gain, the best attribute to split the data is selected.
2. **Splitting the Dataset:** The dataset is split into subsets based on the selected attribute.
3. **Repeating the Process:** The process is repeated recursively for each subset, creating a new internal node or leaf node until a stopping criterion is met (e.g., all instances in a node belong to the same class or a predefined depth is reached).

5.2.3 Metrics for Splitting

Gini Impurity: Measures the likelihood of an incorrect classification of a new instance if it was randomly classified according to the distribution of classes in the dataset.

- $Gini = \sum_{i=1}^n (p_i)^2$, where p_i is the probability of an instance being classified into a particular class.

Entropy: Measures the amount of uncertainty or impurity in the dataset.

- Entropy = $-\sum_{i=1}^n (p_i) \log_2(p_i)$ where p_i is the probability of an instance being classified into a particular class.

Information Gain: Measures the reduction in entropy or Gini impurity after a dataset is split on an attribute.

- InformationGain = Entropy_{parent} - $\sum_{i=1}^n \left(\frac{|D_i|}{|D|} * \text{Entropy}(D_i) \right)$, where D_i is the subset of D after splitting by an attribute.

5.2.4 Performance Analysis

Decision Trees are interpretable models that divide the feature space into hierarchical regions. Similar to Logistic Regression, DT utilized under sampling to address the dataset imbalance. It yielded an accuracy of 90.86% and an F1-score of 90.63%, reflecting its ability to identify patterns effectively. However, its performance lagged slightly behind other ensemble-based methods due to overfitting risks.

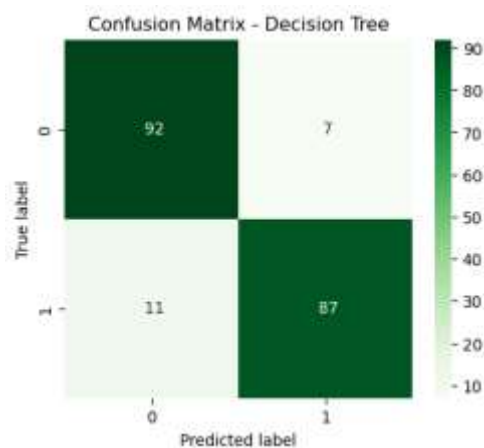


Fig:5.2.1 Confusion Matrix

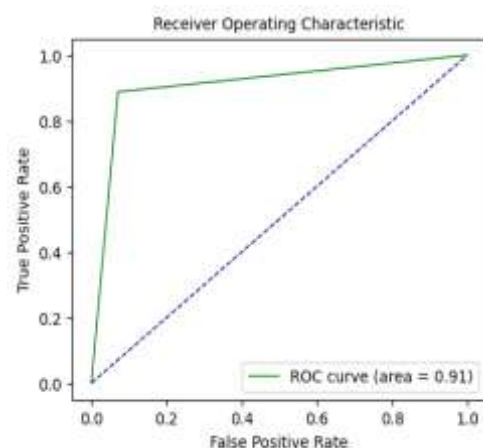


Fig:5.2.2 ROC Curve

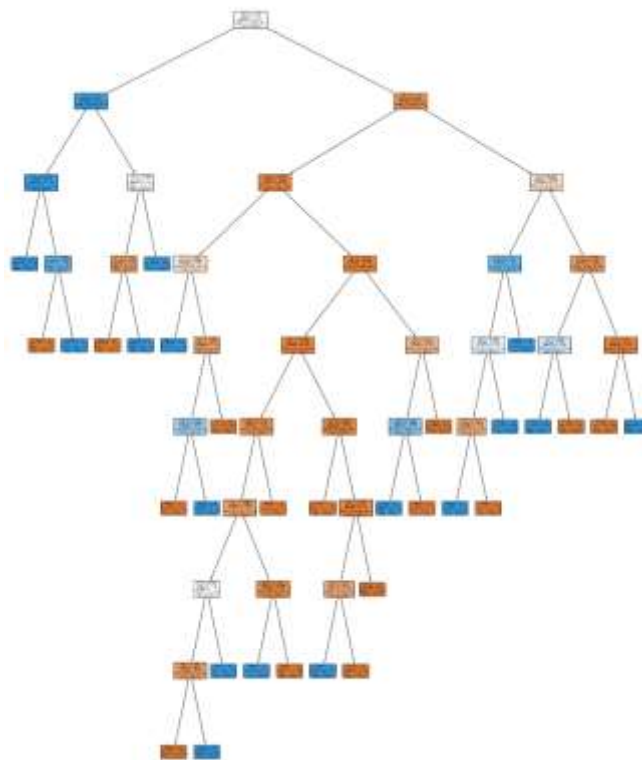


Fig:5.2.3: Decision Tree

5.3 Random Forest

Random Forest algorithm is a powerful tree learning technique in Machine Learning. It works by creating a number of Decision Trees during the training phase. Each tree is constructed using a random subset of the data set to measure a random subset of features in each partition. This randomness introduces variability among individual trees, reducing the risk of overfitting and improving overall prediction performance.

In prediction, the algorithm aggregates the results of all trees, either by voting (for classification tasks) or by averaging (for regression tasks). This collaborative decision-making process, supported by multiple trees with their insights, provides an example stable and precise results. Random forests are widely used for classification and regression functions, which are known for their ability to handle complex data, reduce overfitting, and provide reliable forecasts in different environments.

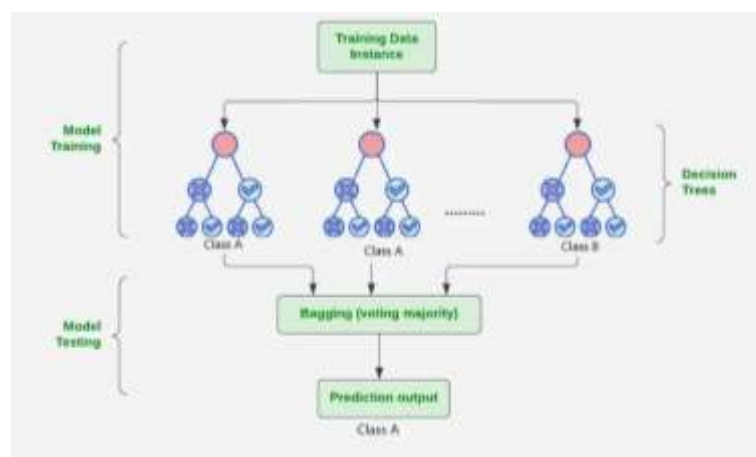


Fig:5.3.1 Random Forest Algorithm in Machine Learning

5.3.1 Algorithm for Random Forest

- **Step 1:** Select random K data points from the training set.
- **Step 2:** Build the decision trees associated with the selected data points (Subsets).
- **Step 3:** Choose the number N for decision trees that you want to build.
- **Step 4:** Repeat Step 1 and 2.
- **Step 5:** For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

5.3.2 How Does Random Forest Works?

The random Forest algorithm works in several steps which are discussed below:

Ensemble of Decision Trees: Random Forest leverages the power of ensemble learning by constructing an army of Decision Trees. These trees are like individual experts, each specializing in a particular aspect of the data. Importantly, they operate independently, minimizing the risk of the model being overly influenced by the nuances of a single tree.

Random Feature Selection: To ensure that each decision tree in the ensemble brings a unique perspective, Random Forest employs random feature selection. During the training of each tree, a random subset of features is chosen. This randomness ensures that each tree focuses on different aspects of the data, fostering a diverse set of predictors within the ensemble.

Bootstrap Aggregating or Bagging: The technique of bagging is a cornerstone of Random Forest's training strategy which involves creating multiple bootstrap samples from the original dataset, allowing instances to be sampled with replacement. This results in different subsets of data for each decision tree, introducing variability in the training process and making the model more robust.

Decision Making and Voting: When it comes to making predictions, each decision tree in the Random Forest casts its vote. For classification tasks, the final prediction is determined by the mode (most frequent prediction) across all the trees. In regression tasks, the average of the individual tree predictions is taken. This internal voting mechanism ensures a balanced and collective decision-making process.

5.3.3 Key Feature of Random Forest

Some of the Key Features of Random Forest are discussed below:

High Predictive Accuracy: Imagine Random Forest as a team of decision-making wizards. Each wizard (decision tree) looks at a part of the problem, and together, they weave their insights into a powerful prediction tapestry. This teamwork often results in a more accurate model than what a single wizard could achieve.

Resistance to Overfitting: Random Forest is like a cool-headed mentor guiding its apprentices (decision trees). Instead of letting each apprentice memorize every detail of their training, it encourages a more well-rounded understanding. This approach helps prevent getting too caught up with the training data which makes the model less prone to overfitting.

Large Datasets Handling: Dealing with a mountain of data? Random Forest tackles it like a seasoned explorer with a team of helpers (decision trees). Each helper takes on a part of the dataset, ensuring that the expedition is not only thorough but also surprisingly quick.

Variable Importance Assessment: Think of Random Forest as a detective at a crime scene, figuring out which clues (features) matter the most. It assesses the importance of each clue in solving the case, helping you focus on the key elements that drive predictions.

Built-in Cross-Validation: Random Forest is like having a personal coach that keeps you in check. As it trains each decision tree, it also sets aside a secret group of cases (out-of-bag) for testing. This built-in validation ensures your model doesn't just ace the training but also performs well on new challenges.

Handling Missing Values: Life is full of uncertainties, just like datasets with missing values. Random Forest is the friend who adapts to the situation, making predictions using the information available. It doesn't get flustered by missing pieces; instead, it focuses on what it can confidently tell us.

Parallelization for Speed: Random Forest is your time-saving buddy. Picture each decision tree as a worker tackling a piece of a puzzle simultaneously. This parallel approach taps into the power of modern tech, making the whole process faster and more efficient for handling large-scale projects.

5.3.4 Performance Analysis

Random Forest is an ensemble of Decision Trees that enhances performance through bagging and feature randomness. This algorithm delivered the highest accuracy of 99.96% but had a lower F1-score of 86.81%, suggesting potential challenges in correctly classifying the minority fraud cases. Despite this, RF excelled in capturing overall patterns in the data.

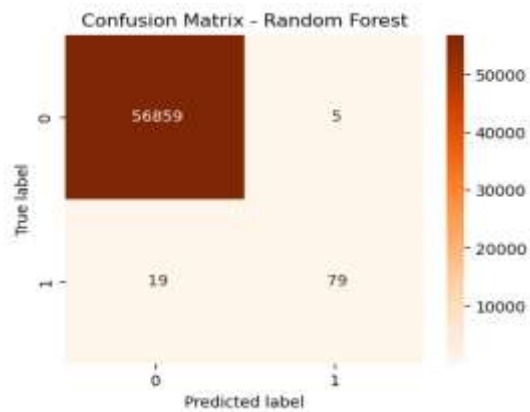


Fig:5.3.2 Confusion Matrix

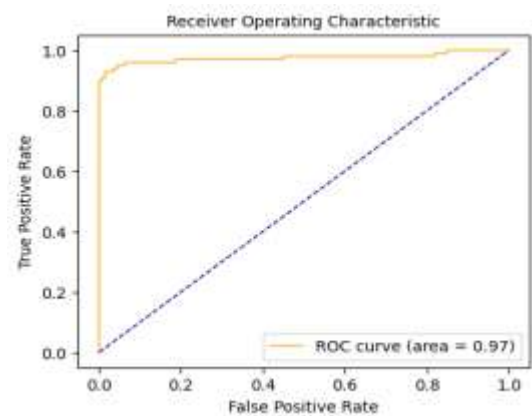


Fig:5.3.3 ROC Curve

Classification Report:					
	precision	recall	f1-score	support	
0	0.89	0.93	0.91	99	
1	0.93	0.89	0.91	98	
accuracy			0.91	197	
macro avg	0.91	0.91	0.91	197	
weighted avg	0.91	0.91	0.91	197	

Fig:5.3.4 Classification Report

5.4 Gradient Boosting

Gradient Boosting is a powerful boosting algorithm that combines several weak learners into strong learners, in which each new model is trained to minimize the loss function such as mean squared error or cross-entropy of the previous model using gradient descent. In each iteration, the algorithm computes the gradient of the loss function with respect to the predictions of the current ensemble and then trains a new weak model to minimize this gradient. The predictions of the new model are then added to the ensemble, and the process is repeated until a stopping criterion is met.

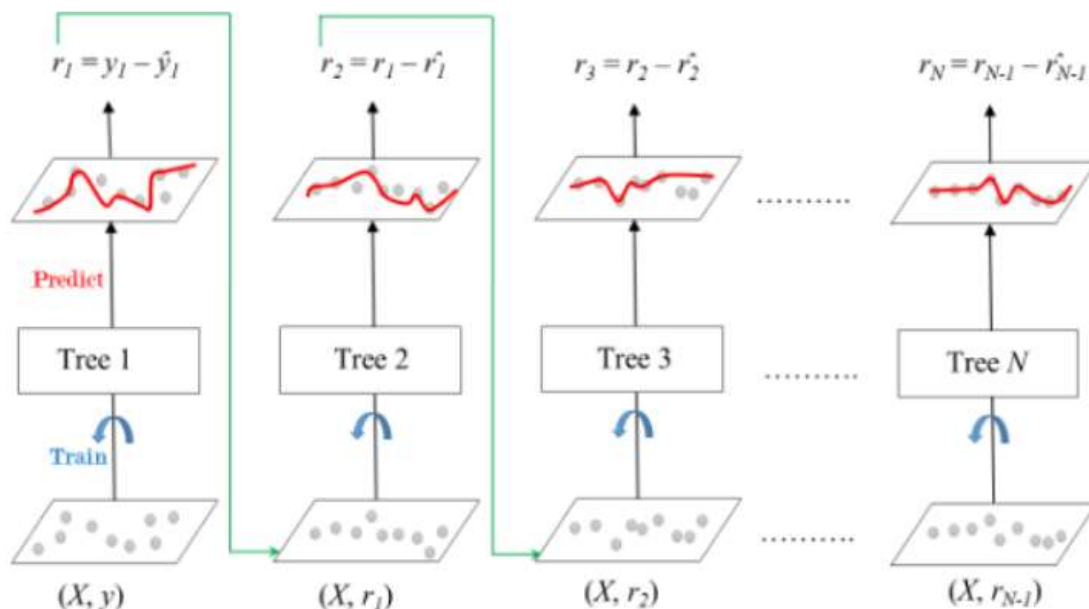


Fig:5.4.1 Gradient Boosted Trees for Regression

The ensemble consists of M trees. Tree1 is trained using the feature matrix X and the labels y . The predictions labelled $\hat{y}_1$ are used to determine the training set residual errors r_1 . Tree2 is then trained using the feature matrix X and the residual errors r_1 of Tree1 as labels. The predicted results $\hat{r}_1$ are then used to determine the residual r_2 . The process is repeated until all the M trees forming the ensemble are trained. There is an important parameter used in this technique known as Shrinkage. Shrinkage refers to the fact that the prediction of each tree in the ensemble is shrunk after it is multiplied by the learning rate (η) which ranges between 0 to 1. There is a trade-off between η and the number of estimators, decreasing learning rate needs to be compensated with increasing estimators in order to reach certain model

performance. Since all trees are trained now, predictions can be made. Each tree predicts a label and the final prediction is given by the formula,

$$y(\text{pred}) = y_1 + (\text{eta} * r_1) + (\text{eta} * r_2) + \dots + (\text{eta} * r_N)$$

5.4.1 Gradient Boosting Algorithm

Step 1: Let's assume X, and Y are the input and target having N samples. Our goal is to learn the function $f(x)$ that maps the input features X to the target variables y. It is boosted trees i.e the sum of trees.

The loss function is the difference between the actual and the predicted variables.

$$L(f) = \sum_{i=1}^N L(y_i, f(x_i))$$

Step 2: We want to minimize the loss function $L(f)$ with respect to f .

$$\hat{f}_0(x) = \underset{f}{\operatorname{argmin}} L(f) = \underset{f}{\operatorname{argmin}} \sum_{i=1}^N L(y_i, f(x_i))$$

If our gradient boosting algorithm is in M stages then to improve the f_m the algorithm can add some new estimator as h_m having $1 \leq m \leq M$

$$\hat{y}_i = F_{m+1}(x_i) = F_m(x_i) + h_m(x_i)$$

Step 3: Steepest Descent

For M stage gradient boosting, The steepest Descent finds $h_m = -\beta_m g_m$ where β_m is constant and known as step length and g_m is the gradient of loss function $L(f)$.

$$g_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x_i)=f_{m-1}(x_i)}$$

Step 4: Solution

The gradient Similarly for M trees:

$$f_m(x) = f_{m-1}(x) + \left(\underset{h_m \in H}{\operatorname{argmin}} \left[\sum_{i=1}^N L(y_i, f_{m-1}(x_i) + h_m(x_i)) \right] \right) (x)$$

The current solution will be

$$f_m = f_{m-1} - \rho_m g_m$$

5.4.2 Performance Analysis

Gradient Boosting is another ensemble method that builds trees sequentially, minimizing errors iteratively. Using the sklearn implementation, GB achieved an accuracy of 99.94% but had a relatively lower F1-score of 79.31%, highlighting its strength in capturing majority class patterns while occasionally misclassifying fraud cases.

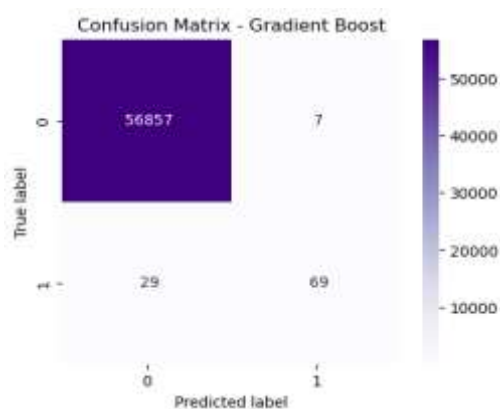


Fig:5.4.2 Confusion Matrix

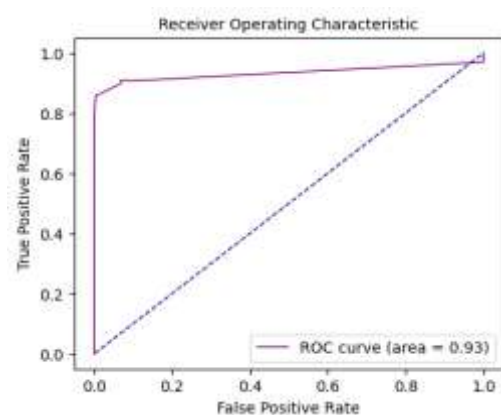


Fig:5.4.3 ROC Curve

Classification Report:				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.91	0.70	0.79	98
accuracy			1.00	56962
macro avg	0.95	0.85	0.90	56962
weighted avg	1.00	1.00	1.00	56962

Fig:5.4.4 Classification Report

5.5 Confusion Matrix and ROC Curve Analysis

To thoroughly evaluate the performance of our machine learning models for credit card fraud detection, we analysed the Confusion Matrix and Receiver Operating Characteristic (ROC) Curve for each algorithm. These tools provided a deeper understanding of how well the models distinguish between fraudulent and non-fraudulent transactions.

5.5.1 Confusion Matrix

The Confusion Matrix is a tabular representation that outlines the performance of a classification algorithm by displaying the counts of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

True Positives (TP): Transactions correctly identified as fraud.

True Negatives (TN): Transactions correctly identified as non-fraud.

False Positives (FP): Non-fraudulent transactions misclassified as fraud.

False Negatives (FN): Fraudulent transactions misclassified as non-fraud.

Analysing these metrics helped us understand each algorithm's ability to minimize false alarms (FP) and correctly identify fraudulent transactions (TP). For example, Logistic Regression showed a well-balanced confusion matrix with low FP and FN values, indicating a strong ability to classify both classes effectively.

5.5.2 ROC Curve

The ROC Curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) across various threshold values. This curve illustrates the trade-off between sensitivity and specificity for a classification model. The Area Under the Curve (AUC) provides a single value summarizing the model's ability to distinguish between classes:

AUC closer to 1: Indicates a highly effective model.

AUC closer to 0.5: Indicates a model with no discriminative power.

For our project:

Logistic Regression achieved a well-balanced ROC curve with a high AUC, reflecting its ability to maintain a consistent trade-off.

Random Forest and Gradient Boosting achieved nearly perfect ROC curves with high AUC values, signifying their strong discriminatory power despite their occasional misclassification of minority fraud cases.

Decision Tree displayed a reasonable ROC curve, but its AUC was slightly lower than the ensemble models, reflecting its lower accuracy and susceptibility to overfitting.

5.6 Performance Comparison

The algorithms were evaluated holistically based on their accuracy, F1-score, and robustness to the imbalanced dataset. Logistic Regression excelled in achieving a balanced performance across metrics. While Random Forest and Gradient Boosting demonstrated exceptional accuracy, their lower F1-scores indicated that they were less effective in managing the minority class.

5.6.1 Accuracy Table

Models	Accuracy	F1-Score
Logistic Regression	93.90%	0.93
Decision Tree	90.86%	0.90
Random Forest	99.95%	0.86
Gradient Boosting	99.93%	0.79

5.6.1 Comparison of Model Metrics

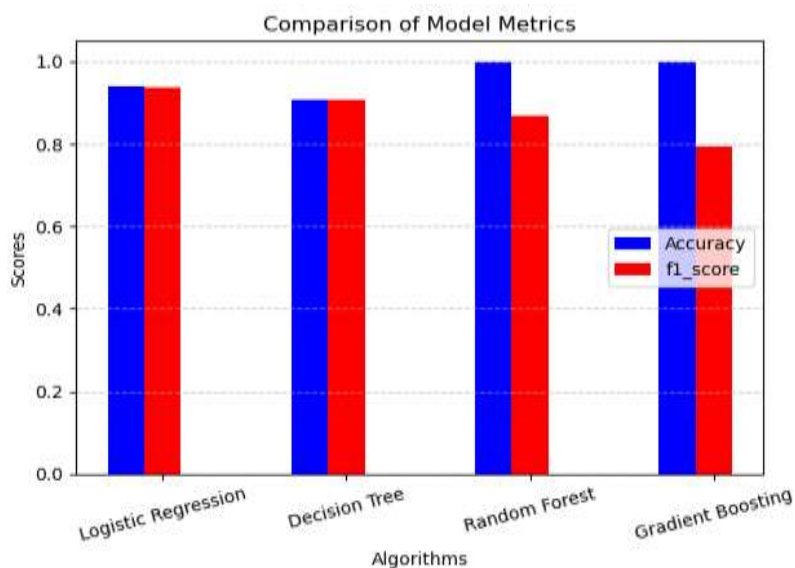


Fig:5.6.1 Comparison of Model Metrics

CHAPTER 6

Future Enhancements and Conclusion

6.1 Future Enhancements

The credit card fraud detection system developed in this project demonstrates significant progress in identifying fraudulent transactions using machine learning algorithms. However, there is ample scope for further enhancement. Future work could focus on the following areas:

Real-time Implementation: Deploy the system in a real-time environment to evaluate its performance on live data streams and optimize it for faster predictions.

Advanced Algorithms: Incorporate deep learning techniques like recurrent neural networks (RNN) or transformers, which can capture sequential transaction patterns and improve fraud detection accuracy.

Feature Engineering: Perform in-depth feature extraction and engineering to identify hidden patterns or transaction behaviours that could enhance model performance.

Handling Data Imbalance: Experiment with advanced sampling techniques or cost-sensitive algorithms to address the challenges of imbalanced datasets effectively.

Explain ability: Integrate explainable AI (XAI) techniques to make the system more transparent and understandable for stakeholders, enabling better trust and accountability.

Cross-Domain Generalization: Test the models on datasets from other financial domains or regions to evaluate their generalization capability.

By addressing these aspects, the system can be made more robust, scalable, and adaptable to evolving fraud patterns.

6.2 Conclusion

This project explored the application of machine learning algorithms for credit card fraud detection using Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. The analysis demonstrated the significance of balancing datasets and evaluating performance metrics beyond accuracy, such as precision, recall, F1-score, and ROC-AUC.

Logistic Regression emerged as the most balanced algorithm, providing consistent results across all evaluation criteria. While Random Forest and Gradient Boosting achieved exceptional accuracy, their performance in handling imbalanced data revealed room for improvement. The use of confusion matrices and ROC curves provided valuable insights into the practical implications of the models' predictions.

REFERENCES

- [1] Adepoju, O., Wosowei, J., Lawte, S., & Jaiman, H.: "Credit Card Fraud Detection Using Machine Learning." This study compared the effectiveness of KNN, SVM, and Logistic Regression for fraud detection. The models were trained on a credit card transaction dataset to classify fraudulent and non-fraudulent transactions. Published in repository.rit.edu.
- [2] Ryman-Tubb, N. F., Krause, P., & Garn, W.: "A Machine Learning Based Credit Card Fraud Detection Using the GA." This paper proposed a genetic algorithm-based feature selection method combined with machine learning models for fraud detection. Published in the Journal of Big Data, 2022.
- [3] Talukder, M. A., Hossen, R., Uddin, M. A., Uddin, M. N., & Acharjee, U.: "Securing Transactions: A Hybrid Dependable Ensemble Machine Learning Model using IHT-LR and Grid Search." This paper introduced a hybrid ensemble model integrating Decision Tree, Random Forest, KNN, and MLP for fraud detection. Published on arXiv, 2024.
- [4] de Souza, D. H. M., & Bordin Jr, C. J.: "Ensemble and Mixed Learning Techniques for Credit Card Fraud Detection." This study evaluated mixed learning methods with K-means preprocessing and detector ensemble techniques for fraud detection. Published on arXiv, 2021.
- [5] Zaffar, Z., Sohrab, F., Kannianen, J., & Gabbouj, M.: "Credit Card Fraud Detection with Subspace Learning-based One-Class Classification." The research focused on handling high-dimensional and imbalanced data using subspace learning techniques. Published on arXiv, 2023.
- [6] D. H. M. de Souza and C. J. Bordin Jr.: "Ensemble and Mixed Learning Techniques for Credit Card Fraud Detection." Evaluates K-means preprocessing and detector ensemble techniques for fraud detection. Published on arXiv, 2021.
- [7] Z. Zaffar, F. Sohrab, J. Kannianen, and M. Gabbouj: "Credit Card Fraud Detection with Subspace Learning-based One-Class Classification." This research focuses on handling high-dimensional and imbalanced data using subspace learning techniques. Published on arXiv, September 2023.